

Simple Inheritance

```
package javacore;

class Parent {

    void displayParent() {

        System.out.println("This is the parent class");

    }

}
```

```
class Child extends Parent {

    void displayChild() {

        System.out.println("This is the child class");

    }

}
```

```
public class SimpleInheritance {

    public static void main(String[] args) {

        Child c = new Child();

        c.displayParent();

        c.displayChild();

    }

}
```

Execution Steps:

1. **Start** the program.
2. JVM executes the **main() method**.
3. Child c = new Child(); creates an object of Child.
4. c.displayParent(); calls the method inherited from Parent.
5. It prints "**This is the parent class**".

6. c.displayChild(); calls the method of Child.

7. It prints **"This is the child class"**.

8. The program **ends**.

Output:

This is the parent class

This is the child class

Multiple Inheritance:

```
package javacore;
```

```
interface Father {
```

```
    void fatherProperty();
```

```
}
```

```
interface Mother {
```

```
    void motherProperty();
```

```
}
```

```
class Child implements Father, Mother {
```

```
    public void fatherProperty() {
```

```
        System.out.println("Child gets property from Father");
```

```
    }
```

```
    public void motherProperty() {
```

```
        System.out.println("Child gets property from Mother");
```

```
    }
```

```
    void ownProperty() {
```

```
        System.out.println("Child has own property");
```

```

    }
}

public class MultipleInheritance {

    public static void main(String[] args) {

        Child mi = new Child();

        mi.fatherProperty();
        mi.motherProperty();
        mi.ownProperty();
    }
}

```

Execution Steps:

START

JVM starts execution

main() method is called

Child mi = new Child();

Child object 'mi' is created

mi.fatherProperty();

Child's fatherProperty() method executes

Output: Child gets property from Father

mi.motherProperty();

Child's motherProperty() method executes

Output: Child gets property from Mother

mi.ownProperty();

Child's ownProperty() method executes

Output: Child has own property

END

OUTPUT:

Child gets property from Father

Child gets property from Mother

Child has own property

MultilevelInheritance

```
package javacore;
```

```
class Grandparent {  
    void displayGrandparent() {  
        System.out.println("This is the grandparent class");  
    }  
}
```

```
class Parent extends Grandparent {  
    void displayParent() {  
        System.out.println("This is the parent class");  
    }  
}
```

```
class Child extends Parent {  
    void displayChild() {  
        System.out.println("This is the child class");  
    }  
}
```

```
public class MultilevelInheritance {  
    public static void main(String[] args) {
```

```
Child c = new Child();

c.displayGrandparent();

c.displayParent();

c.displayChild();

}

}
```

Execution Steps

1. Start the program.

2. Java enters the main() method.

3. Create an object of Child class:

```
Child c = new Child();
```

4. Since Child extends Parent and Parent extends Grandparent, the Child object can access methods of all three classes.

5. Call:

```
c.displayGrandparent();
```

Output:

This is the grandparent class

6. Call:

```
c.displayParent();
```

Output:

This is the parent class

7. Call:

```
c.displayChild();
```

Output:

This is the child class

8. Program execution ends.

Output:

This is the grandparent class

This is the parent class

This is the child class

Hybrid Inheritance

```
package javacore;

class Parent {
    void displayParent() {
        System.out.println("This is the parent class");
    }
}

class Child1 extends Parent {
    void displayChild1() {
        System.out.println("This is Child 1");
    }
}

interface Child2 {
    void displayChild2();
}

class GrandChild extends Child1 implements Child2 {
```

```

    public void displayChild2() {
        System.out.println("This is Child 2");
    }

    void displayGrandChild() {
        System.out.println("This is the grandchild class");
    }
}

public class HybridInheritance {
    public static void main(String[] args) {

        GrandChild obj = new GrandChild();

        obj.displayParent();
        obj.displayChild1();
        obj.displayChild2();
        obj.displayGrandChild();
    }
}

```

Execution Steps

1. Start the program.
2. The main() method starts execution.
3. A GrandChild object is created:


```
GrandChild obj = new GrandChild();
```
4. GrandChild extends Child1.

Child1 extends Parent.

Therefore, GrandChild inherits methods from:

Parent → Child1 → GrandChild

5. GrandChild also implements the Child2 interface.

Therefore, it must provide the displayChild2() method.

6. The statement

```
obj.displayParent();
```

calls the displayParent() method of Parent class.

7. The statement

```
obj.displayChild1();
```

calls the displayChild1() method of Child1 class.

8. The statement

```
obj.displayChild2();
```

calls the displayChild2() method implemented from Child2 interface.

9. The statement

```
obj.displayGrandChild();
```

calls the displayGrandChild() method of GrandChild class.

10. All four messages are displayed.

11. Program execution ends.

Output

This is the parent class

This is Child 1

This is Child 2

This is the grandchild class

Hierarchial Inheritance

```
package javacore;

class Parent {

    void displayParent() {

        System.out.println("This is the parent class");

    }

}

class Child1 extends Parent {

    void displayChild1() {

        System.out.println("This is Child 1");

    }

}

class Child2 extends Parent {

    void displayChild2() {

        System.out.println("This is Child 2");

    }

}

public class HierarchialInheritance {

    public static void main(String[] args) {

        Child1 c1 = new Child1();

        Child2 c2 = new Child2();

    }

}
```

```
c1.displayParent();
c1.displayChild1();

c2.displayParent();
c2.displayChild2();
}
}
```

Execution Steps

1. Start the program.
2. The main() method starts execution.
3. A Child1 object is created:
`Child1 c1 = new Child1();`
4. A Child2 object is created:
`Child2 c2 = new Child2();`
5. Child1 extends Parent, so c1 can access
the displayParent() method.
6. The statement
`c1.displayParent();`
calls the displayParent() method of Parent class.
7. The statement
`c1.displayChild1();`
calls the displayChild1() method of Child1 class.

8. Child2 also extends Parent, so c2 can access the displayParent() method.

9. The statement

```
c2.displayParent();
```

calls the displayParent() method of Parent class.

10. The statement

```
c2.displayChild2();
```

calls the displayChild2() method of Child2 class.

11. All the messages are displayed.

12. Program execution ends.

Output

This is the parent class

This is Child 1

This is the parent class

This is Child 2